

Aula 7 – Vetores e Prática de Programação

Capacitação em Desenvolvimento Full Stack

Lógica de Programação e Algoritmos – 32h

Turno Noturno -30/07/2026 a 10/08/2026 - 18h30h às 21h30h

Professora Esp.: Hélida Dias Batista Xerfan

“Não é a linguagem de programação que define o programador, mas sim sua lógica.” David Ribeiro Guilherme



O que veremos hoje:

- Recapitulação da aula 6
- Vetores
- Prática de programação

Recapitulando a aula 6

- ✓ Repetição com teste no início (ENQUANTO)
- ✓ Repetição com teste no final (REPITA)
- ✓ Repetição contada (PARA)

VETOR

COMO UM GRANDE GAVETEIRO



Um **vetor** (ou array) é como um grande gaveteiro: um único móvel com várias gavetas separadas.



Cada gaveta pode guardar um valor diferente, mas do **mesmo tipo**!

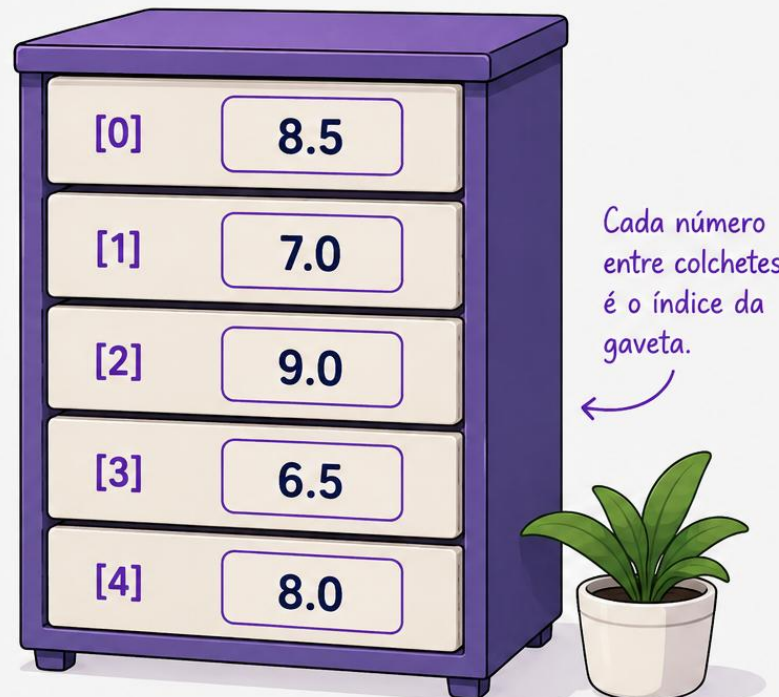


Para achar um valor específico, basta sabermos o número da gaveta (**índice**).



IMPORTANTE!

Todas as posições armazenam valores do **mesmo tipo**.



notas					
↓					
8.5	7.0	9.0	6.5	8.0	← valores
↑	↑	↑	↑	↑	
0	1	2	3	4	← índices

POR QUE PRECISAMOS DE VETORES?

Porque lidar com muitos dados de forma **organizada e eficiente** ficaria difícil (ou até impossível) sem eles!

✗ SEM VETORES

Para guardar 5 notas, precisaríamos criar 5 variáveis diferentes:

nota1 = 8.5
nota2 = 7.0
nota3 = 9.0
nota4 = 6.5
nota5 = 8.0



- ✗ Código maior e repetitivo
- ✗ Difícil de manter e atualizar
- ✗ Difícil de percorrer e processar os dados
- ✗ Risco maior de erros

✓ COM VETORES

Com um vetor, guardamos tudo em uma **única estrutura**:

notas = [8.5, 7.0, 9.0, 6.5, 8.0]

notas →	8.5	7.0	9.0	6.5	8.0
	↑	↑	↑	↑	↑
índices →	0	1	2	3	4

- ✓ Código menor e mais legível
- ✓ Fácil de manter e atualizar
- ✓ Fácil de percorrer e processar os dados
- ✓ Menos erros, mais produtividade

VETORES ESTÃO PRESENTES EM MUITAS SITUAÇÕES DO DIA A DIA



Listas de nomes,
produtos, tarefas



Notas de alunos
e resultados



Leituras de sensores,
temperaturas, medições



Dias da semana,
meses do ano



Pontuações de jogos,
rankings e estatísticas

POR QUE PRECISAMOS DE VETORES?

Porque lidar com muitos dados de forma **organizada e eficiente** ficaria difícil (ou até impossível) sem eles!

✗ SEM VETORES

Para guardar 5 notas, precisaríamos criar 5 variáveis diferentes:

nota1 = 8.5
nota2 = 7.0
nota3 = 9.0
nota4 = 6.5
nota5 = 8.0



- ✗ Código maior e repetitivo
- ✗ Difícil de manter e atualizar
- ✗ Difícil de percorrer e processar os dados
- ✗ Risco maior de erros

✓ COM VETORES

Com um vetor, guardamos tudo em uma **única estrutura**:

notas = [8.5, 7.0, 9.0, 6.5, 8.0]

notas →	8.5	7.0	9.0	6.5	8.0
	↑	↑	↑	↑	↑
índices →	0	1	2	3	4

- ✓ Código menor e mais legível
- ✓ Fácil de manter e atualizar
- ✓ Fácil de percorrer e processar os dados
- ✓ Menos erros, mais produtividade

VETORES ESTÃO PRESENTES EM MUITAS SITUAÇÕES DO DIA A DIA



Listas de nomes,
produtos, tarefas



Notas de alunos
e resultados



Leituras de sensores,
temperaturas, medições



Dias da semana,
meses do ano



Pontuações de jogos,
rankings e estatísticas



A DUPLA DINÂMICA: VETORES E O LAÇO “PARA”

O laço “para” é o melhor amigo do vetor:
ele percorre todas as posições **automaticamente**!



1. AUTOMATIZAÇÃO

O laço de repetição para (for) é o melhor amigo do vetor. Ele permite percorrer todas as gavetas do vetor de forma totalmente **automática**, sem repetir código.



2. A VARIÁVEL CONTADORA

Nós usamos a variável de controle do laço (geralmente chamada de *i*) para representar o índice. Exemplo mágico: `leia(notas[i])`.



3. EFICIÊNCIA MÁXIMA

Em vez de digitar 10 vezes o comando `leia`, você escreve apenas um bloco `para i de 1 ate 10 faça`, economizando tempo e evitando erros humanos.



o laço “para”
percorre assim: → 0 → 1 → 2 → 3 → 4

Exemplo em Portugol:

`para i de 0 ate 4 faça`

`leia(notas[i])`

`fimpara`

← o laço que controla a repetição

← leitura do valor da posição *i* do vetor

← fim do laço

❌ SEM O LAÇO PARA

```
leia(notas[0])
leia(notas[1])
leia(notas[2])
...
leia(notas[9])
```

Muito código repetitivo e maior risco de erros!

VS

✅ COM O LAÇO PARA

```
para i de 0 ate 9 faça
  leia(notas[i])
fimpara
```

Código simples, limpo e muito mais eficiente!



O laço “para” + vetor = programa organizado, rápido e confiável!

Demonstração – Vetores

```
1 Algoritmo "MediaDaTurma"
2 // Ler 5 notas, calcular a média da turma
3 // e mostrar as notas acima da média
4
5 var
6
7   notas : vetor[0..4] de real    // Vetor de 5 notas
8   i : inteiro // Variável de controle
9   soma, media : real
10
11 Inicio
12
13   // Inicializa a variável da soma das notas
14   soma <- 0
15
16   // Leitura das notas
17   para i de 0 ate 4 faça
18
19     escreva("Digite a nota do aluno ", i + 1, ": ")
20     leia(notas[i])
21
22     soma <- soma + notas[i] // Soma cada nota
23
24   fimpara
25
```

```
25
26   // Calcula a média da turma
27   media <- soma / 5
28
29   escreval("")
30   escreval("Média da turma: ", media:4:1)
31
32   escreval("")
33   escreval("Alunos com nota acima da média:")
34
35   // Percorre o vetor pra mostrar quem ficou acima
36   para i de 0 ate 4 faça
37
38     // Verifica se a nota é maior que a média
39     se notas[i] > media entao
40
41       escreval("Aluno ", i + 1,
42               " - Nota: ", notas[i]:4:1)
43
44     fimse
45
46   fimpara
47
48 FimAlgoritmo
```


Exercícios - Vetores

Exercício 1 — Cadastro de Notas: Crie um algoritmo que declare um vetor para armazenar 5 notas, solicite ao usuário que informe cada nota e ao final, exiba todas as notas cadastradas na ordem em que foram informadas e em ordem decrescente.

Ajuda: 1º faça um laço para ler as notas e guardar no vetor; Depois faça um laço para exibir as notas;

Para ordem decrescente compare cada posição com todas as posições que vêm depois dela (use dois laços: laço externo escolhe a posição atual; o laço interno procura um valor maior nas posições seguintes).

Exercício 2 – Número maior: Crie um algoritmo que declare um vetor com 8 números inteiros, solicite que o usuário informe todos os valores. Ao final, exiba o maior número armazenado e a posição em que ele foi encontrado no vetor.

Ajuda: 1º faça um laço para ler os 8 números e guardar no vetor. Use duas variáveis auxiliares, maior e posicao, pra guardar o maior valor encontrado até agora e em que posição ele apareceu. Comece considerando o primeiro número lido (posição 0) como o maior, assim você não corre o risco de chutar um valor inicial errado. Depois, a cada novo número lido, compare com o maior atual: se for maior, atualize as duas variáveis (maior e posicao)

Exercício 3 — Média da turma: Crie um algoritmo que armazene as notas de 10 alunos em um vetor.

Calcule a média da turma, exiba todas as notas cadastradas, a média da turma, quantos alunos obtiveram nota maior ou igual à média.

Ajuda: 1º faça um laço para ler as 10 notas, guardar cada uma no vetor e, ao mesmo tempo, ir somando todas numa variável soma.

Depois do laço, calcule a média dividindo soma por 10. Faça um segundo laço só pra exibir as notas guardadas no vetor.

Por fim, um terceiro laço que percorre o vetor de novo comparando cada nota com a média: toda vez que a nota for maior ou igual à média, some 1 num contador.



PRÁTICA DE PROGRAMAÇÃO

Chegou a hora de colocar em prática
o que você aprendeu!



PENSE
no problema



PROGrame
a solução



EVOLUA
com a prática



Vamos praticar!

Desafio 1 — Controle de caixa diário: Crie um algoritmo que calcule o saldo final do caixa de uma loja ao fim do expediente. O algoritmo deve receber o saldo inicial do dia, o valor de 3 vendas realizadas e o valor de 2 despesas ou retiradas feitas durante o dia, somar as vendas e subtrair as despesas do saldo inicial. Ao final, exibir o saldo final do caixa.

Desafio 2 — Atendimento por SLA: Crie um algoritmo que classifique o atendimento de um chamado de suporte técnico de acordo com o tempo de resposta, em minutos, informado pelo usuário. Se o tempo for de até 10 minutos, exibir "Dentro do prazo"; se for de 11 a 30 minutos, exibir "Atenção"; se for maior que 30 minutos, exibir "Crítico".

Desafio 3 — Comissão da equipe de vendas: Crie um algoritmo que calcule a comissão de uma equipe de 5 vendedores, sabendo que cada um recebe 5% sobre o valor que vendeu no mês. O algoritmo deve receber o valor vendido por cada um dos 5 vendedores, armazenado em um vetor, calcular e exibir a comissão de cada um, e ao final exibir também o total vendido por toda a equipe.

Desafio 4 — Reposição de estoque: Crie um algoritmo que simule a reposição automática do estoque de um produto. O algoritmo deve receber a quantidade atual em estoque, a quantidade mínima de segurança e a quantidade reposta a cada pedido feito ao fornecedor. Enquanto a quantidade em estoque estiver abaixo da mínima, o algoritmo deve repor o estoque e contar quantos pedidos foram necessários. Ao final, exibir a quantidade final em estoque e o número total de pedidos realizados.

DÚVIDAS / PERGUNTAS ?

Este é o momento para esclarecer suas **dúvidas**
e compartilhar suas **perguntas**!



Fique à vontade!

Nenhuma dúvida é boba quando o assunto
é **aprender e evoluir**.



(92) 99218 – 0056



www.iteam.org.br



Av. Glaycon de Paiva nº 1820,
Mecejana
CEP: 69.304-560
Boa Vista/RR
CNPJ: 48.653.884/0001-00

O que vimos hoje

- ✓ Vetores
- ✓ Prática de programação

Próxima aula (10/08):

- Introdução a Python

Até a próxima aula!

Aula 8 – 10/08/2026 (Segunda-feira) – 18h30 às 21h30